SSR VS CSR EN HUN CLIENT-SIDE FOOTPRINT

Een studie naar de invloed van render-strategie op CPU- en geheugengebruik op mobiele apparaten

Daniël van Ginneken

Avans University of Applied Sciences | Lovensdijkstraat 61, 4818 AJ Breda

Samenvatting

In dit onderzoeksvoorstel ga ik onderzoeken wat de invloed is van server-side rendering (SSR) en client-side rendering (CSR) op het energieverbruik van mobiele apparaten. Hoewel deze render-strategieën vaak worden vergeleken op basis van snelheid en gebruikerservaring, is er minder aandacht voor resource-efficiëntie en energieverbruik. In dit onderzoek ga ik één webapplicatie ontwikkelen in twee varianten (SSR en CSR), waarbij onder gecontroleerde omstandigheden het CPU- en geheugengebruik wordt gemeten tijdens het renderproces. De resultaten worden geanalyseerd om inzicht te krijgen in de verschillen tussen beide strategieën en hun impact op resource-efficiëntie. Daarmee draagt dit onderzoek bij aan het maken van bewustere keuzes rondom render strategieën in mobiele front-end ontwikkeling.

Introductie

Ik ben een grote fan van React, zowel de client-side rendering (CSR) als server-side rendering (SSR) frameworks. Ik vind CSR superhandig vanwege de eenvoud: data ophalen van een externe API is makkelijker en de rendering gebeurt altijd aan de client-side, waardoor ik overal interactiviteit kan toevoegen. Aan de andere kant biedt SSR voordelen zoals betere SEO en mogelijk snellere Time To First Paint (TTFP), waardoor de gebruiker sneller iets ziet bij het laden van de pagina.

Tijdens het coderen aan een eigen project vroeg ik me af welke render strategie ik het beste kon gebruiken in die situatie. De doelgroep van de applicatie is gebruikers die de applicatie langere tijd op hun mobiele apparaten gebruiken, zoals smartphones en tablets. Toen realiseerde ik me dat ik eigenlijk niet weet welke strategie daadwerkelijk minder energie verbruikt.

Hier deed ik wat onderzoek naar maar er was niet veel informatie over het energieverbuik voor beide rendering strategieën. Dit vormde de aanleiding voor mijn onderzoek. Ik wil onderzoeken hoe de keuze tussen SSR en CSR het energieverbruik en het gebruik van CPU en geheugen op mobiele client-apparaten beïnvloedt. Hiermee kan ik niet alleen mijn eigen projecten verbeteren, maar ook andere front-end ontwikkelaars helpen bij het maken van bewuste keuzes rondom energie- en resource-efficiëntie.

Literatuur-review

Er is al veel onderzoek gedaan naar de verschillen tussen client-side rendering en server-side rendering binnen webapplicaties. In de meeste studies ligt de focus op performance in

termen van snelheid en gebruikerservaring. Nordström en Dixelius (2023) laten bijvoorbeeld zien dat SSR bij grotere en complexere webpagina's betere laadprestaties kan leveren dan CSR, terwijl bij kleinere pagina's het verschil minimaal is. Ook Clapton et al. (2025) tonen aan dat SSR vooral bij grotere datavolumes beter scoort op metrics zoals Time To Interactive (TTI) en Largest Contentful Paint (LCP).

Daarnaast beschrijft Ojala (2025) dat SSR voordelen biedt op het gebied van zoekmachineoptimalisatie (SEO) en initiële laadtijd. Tegelijkertijd introduceert SSR extra complexiteit in het ontwikkelproces, bijvoorbeeld door serverbelasting en het gebruik van hydration-mechanismen. Wat mij opvalt in deze onderzoeken is dat performance bijna altijd wordt geïnterpreteerd als snelheid of gebruikerservaring, en minder vaak als efficiënt gebruik van resources.

In enkele studies wordt ook gekeken naar CPU- en geheugengebruik. Gaddam (2025) rapporteert dat SSR gemiddeld leidt tot een lagere CPU- en geheugenbelasting tijdens de renderfase, met name op client apparaten. Dit wordt verklaard doordat een groter deel van de verwerking op de server plaatsvindt, waardoor de client minder JavaScript hoeft uit te voeren. Ook analyseren Clapton et al. (2025) het gebruik van de JavaScript Heap en tonen aan dat bij grotere datasets de client-side belasting bij CSR aanzienlijk toeneemt ten opzichte van SSR. Dit suggereert dat de keuze voor een render strategie niet alleen invloed heeft op snelheid, maar ook op de belasting van het apparaat van de gebruiker.

Wanneer ik kijk naar het energieverbruik aspect van webapplicaties, blijkt dat dit onderwerp minder direct gekoppeld wordt aan specifieke renderstrategieën. Thiagarajan et al. (2012) tonen met een hardwarematige meetopstelling aan dat vooral JavaScript- en CSS-verwerking in de browser een significante invloed hebben op het energieverbruik van mobiele apparaten. Recente studies, zoals Kalliola en Vepsäläinen (2025), geven aan dat het meten van energieverbruik van webapplicaties complex is. Zij stellen dat CPU-gebruik kan dienen als een bruikbare indicator voor energieverbruik, maar dat bestaande online "energy calculators" vaak onnauwkeurige of inconsistente resultaten geven.

Wat in de bestaande literatuur opvalt, is dat resourcegebruik en energie-efficiëntie zelden het primaire onderzoeksdoel vormen. Vaak worden CPU- en geheugenmetingen meegenomen als ondersteunende metrics bij performance-analyses, maar niet als centrale evaluatiecriteria.

Op basis van deze literatuur kan ik concluderen dat SSR en CSR uitgebreid zijn vergeleken op het gebied van snelheid, gebruikerservaring en SEO. Ook is aangetoond dat render-strategieën invloed hebben op CPU- en geheugengebruik op client-apparaten. Er lijkt echter beperkt onderzoek te zijn dat SSR en CSR expliciet vergelijkt met resource-

efficiëntie en specifiek binnen een mobiele context. Dit vormt de aanleiding voor het huidige onderzoek, waarin de nadruk wordt verlegd van “performance als snelheid” naar “performance als resource efficiëntie”.

Onderzoeks-vraag

Na het uitvoeren van het literatuuronderzoek heb ik gezien dat SSR en CSR vooral worden vergeleken op basis van snelheid, gebruikerservaring en SEO. Enkele studies richten zich ook op CPU- en geheugengebruik, maar hier staat resource-efficiëntie niet centraal en meestal ook niet in een mobile device context. Dit onderzoek richt zich daarom op de invloed van render-strategieën op het resourcegebruik van mobile client devices. Dit onderzoek heeft daarom de volgende hoofdvraag:

Hoe beïnvloedt de keuze tussen server-side rendering (SSR) en client-side rendering (CSR) het CPU- en geheugengebruik op mobiele client-apparaten tijdens het renderproces, en wat betekent dit voor het ontwikkelen van resource-efficiënte frontends?

Om deze vraag te beantwoorden, maak ik gebruik van de volgende deelvragen:

1. Wat zijn de technische verschillen tussen SSR en CSR met betrekking tot verwerking en belasting van de client tijdens het renderproces?
2. In welke mate verschilt het gemeten CPU-gebruik tussen SSR en CSR bij identieke webapplicaties op mobiele apparaten?
3. In welke mate verschilt het geheugengebruik tussen SSR en CSR in dezelfde context?
4. In hoeverre kan CPU-gebruik worden gebruikt als indicator voor energieverbruik bij het vergelijken van render-strategieën?

Onderzoeks-opzet en methodes

Om te onderzoeken wat het effect is van SSR en CSR op CPU- en geheugengebruik op mobiele apparaten, kies ik voor een gecontroleerde aanpak met experimenten. Het doel is om zo goed mogelijk te meten wat het verschil is tussen deze 2 render-strategieën, zonder dat andere factoren de resultaten beïnvloeden.

Het onderzoek start met een literatuurstudie om bestaande inzichten over render-strategieën, CPU-gebruik en energieverbruik te analyseren. Dit helpt me om de onderzoeksvraag te onderbouwen en om te bepalen hoe resourcegebruik het best gemeten kan worden. Daarnaast ga ik een probleemanalyse doen om het verschil tussen performance als snelheid en performance als resource-efficiëntie duidelijk te scheiden.

Vervolgens ga ik 1 webapplicatie bouwen in twee varianten: server-side rendering (SSR) en client-side rendering (CSR). Beide applicaties krijgen exact dezelfde functionaliteit, dataset en gebruikersinteractie. Het enige verschil is de render-strategie. Door dit zo op te zetten, wordt één variabele aangepast (SSR of CSR), terwijl de rest gelijk blijft. Dit maakt het mogelijk om het effect van de render-strategie direct te meten.

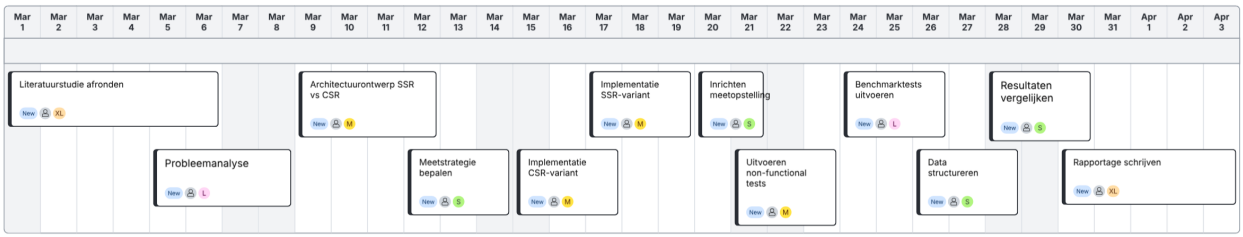
Tijdens de testfase zal ik de applicatie hosten op server binnen hetzelde netwerk waarin een mobiel apparaat zit. Deze ga ik gebruiken om de tests uit te voeren. In de tests ga ik CPU-gebruik en geheugengebruik meten tijdens het initiële renderproces en bij interactie met de applicatie. Dit ga dit meerdere keren moeten doen om uitschieters te beperken. Daarnaast run ik een benchmarktest om de resultaten van SSR en CSR direct met elkaar te vergelijken. De verzamelde data ga ik daarna analyseren, zodat gemiddelde verschillen en patronen zichtbaar worden.

Deze aanpak is voor mij best uitvoerbaar binnen ~5 weken, omdat ik het onderzoek richt op één applicatie, een specifieke metrics (CPU, geheugen en energieverbruik) en één type apparaat. Door het onderzoek af te bakenen tot het renderproces en niet de volledige levenscyclus van een applicatie, blijft het haalbaar en overzichtelijk.

Fase	***Methode***	***Type***	***Waarom gebruikt?***
Oriëntatie	Literature study	Library	Onderbouwen van SSR/CSR
Oriëntatie	Best good and bad practices	Library	Inzicht krijgen in bestaande optimalisatiestrategieën
Analyse	Problem analysis	Field	Performance vs resource-efficiëntie
Analyse	Domain modelling	Field	Modelleren van verschil tussen architectuur
Ontwerp	Prototyping	Workshop	Bouwen van de applicatie
Testfase	Non-functional test	Lab	Meten van CPU- en geheugengebruik

Testfase	Benchmark test	Showroom	Direct vergelijken van SSR vs CSR
Analyse	Descriptive statistics	Field	Gemiddelden en verschillen berekenen
Validatie	Peer review	Showroom	Controleren van methode en interpretatie

Planning



Het onderzoek moet worden uitgevoerd over een periode van ongeveer 5 weken. In de eerste week ga ik de literatuurstudie afronden. Ook zal ik een probleemanalyse uitvoeren om sommige onderdelen van het onderzoek af te bakenen.

In week twee ga ik de applicatie uitwerken. Hierbij zal ik de architectuur van zowel de SSR- als CSR-variant moeten documenteren en ga ik de meetstrategie bepalen. Daarnaast wordt de testomgeving voorbereid.

Week drie staat in het teken van implementatie. Hier ga ik de applicatie ontwikkelen in twee varianten (SSR en CSR).

In week vier ga ik metingen uitvoeren. De applicaties worden getest op een mobiel apparaat onder gecontroleerde omstandigheden. De metingen worden meerdere keren herhaald om de betrouwbaarheid te vergroten.

In week vijf ga ik de verzamelde data analyseren en de resultaten van SSR en CSR met elkaar vergeleken. Daarna wordt het tijd voor het formuleren van conclusies, het beantwoorden van de hoofd- en deelvragen en het afronden van het onderzoeksrapport.

Verwachte uitkomst

Op basis van de literatuur en technische analyse denk ik dat server-side rendering gemiddeld energiezuiniger zal zijn op mobiele apparaten vergeleken met client-side rendering. De belangrijkste reden hiervoor is dat bij SSR het grootste deel van de HTML op

de server wordt gegenereerd, waardoor de client deze alleen maar hoeft te parsen en weer te geven. Bij CSR moet daarentegen eerst JavaScript worden uitgevoerd om de HTML dynamisch te genereren en in de DOM te plaatsen.

Tegelijkertijd is het niet uitgesloten dat CSR in bepaalde situaties efficiënter kan blijken. Bij SSR wordt vaak extra data meegestuurd vanuit de server, bijvoorbeeld in de vorm van geserialiseerde JSON-objecten (zoals bij het React-inflight-protocol in Next.js). Dit kan zorgen voor extra netwerkverkeer en aanvullende verwerking aan de clientzijde. Daarnaast kan de netwerkarchitectuur invloed hebben op de resultaten: bij SSR bevindt de applicatieserver zich vaak dicht bij de database, waardoor data sneller kan worden opgehaald. Bij CSR is er een aparte API-call nodig vanaf de client, wat extra latency en verwerking kan introduceren.

Aan de andere kant biedt CSR voordelen op het gebied van hostingarchitectuur. Een volledig client-side applicatie kan statisch worden gehost via een CDN, waardoor er geen server-side compute nodig is. Dit verlaagt hostingkosten en verschuift de verwerkingslast volledig naar de client.

In het verwachte resultaat van dit onderzoek verwacht ik dat meerdere aspecten naar voren komen. Het gaat daarom niet alleen om een vergelijking in CPU- en geheugengebruik tussen SSR en CSR, maar ook om een beter inzicht in de onderliggende trade-offs tussen serverbelasting, clientbelasting en netwerkverkeer. Ik verwacht dat de uitkomsten frontend-ontwikkelaars kunnen helpen bij het maken van een bewuste keuze voor een render-strategie op basis van resource-efficiëntie in plaats van uitsluitend snelheid. Daarmee kan dit onderzoek bijdragen aan het ontwikkelen van energiezuinigere en duurzamere webapplicaties.

Bronnen

Clapton, J., Bastos, H., Batisteli, J. P. O., Seufitelli, D. B., & Tavares, C. (2025). *Client-Side vs Server-Side Rendering: Impacts on Performance and User Experience in Web Applications*. In *Simpósio Brasileiro de Qualidade de Software (SBQS)* (pp. 165–172). SBC.

Gaddam, R. R. (2025). Innovations in Server-Side Rendering: Boosting SEO and User Engagement. *Journal of Multidisciplinary, 5*(7), 524–529.

Jamali, I. (2022). *Identifying trade-offs associated with cross-platform mobile development tools* (Master's thesis). University of Saskatchewan.

Kalliola, J., & Vepsäläinen, J. (2025). Challenges related to approximating the energy consumption of a website. *IEEE Access*.

Nordström, C., & Dixelius, A. (2023). *Comparisons of server-side rendering and client-side rendering for web pages*.

Ojala, T. (2025). *Server-side rendering in web development*.

Thiagarajan, N., Aggarwal, G., Nicoara, A., Boneh, D., & Singh, J. P. (2012). Who killed my battery? Analyzing mobile browser energy consumption. In *Proceedings of the 21st International Conference on World Wide Web* (pp. 41–50).